Computer Organization and Architecture [ENCT 303]

Chapter 1: Introduction

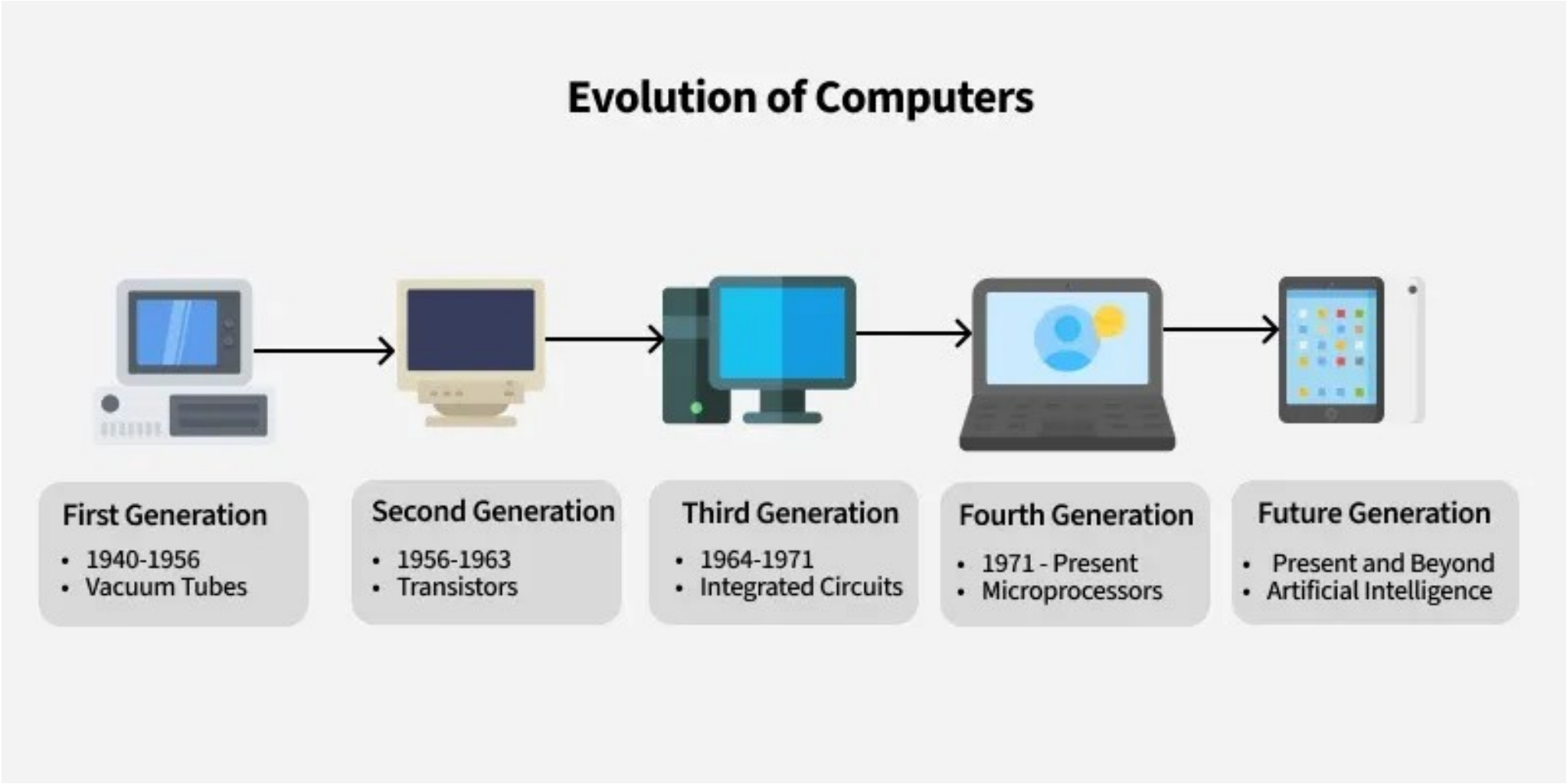
Bachelor in Computer Engineering (BCT)

Dr. Binayak Ojha

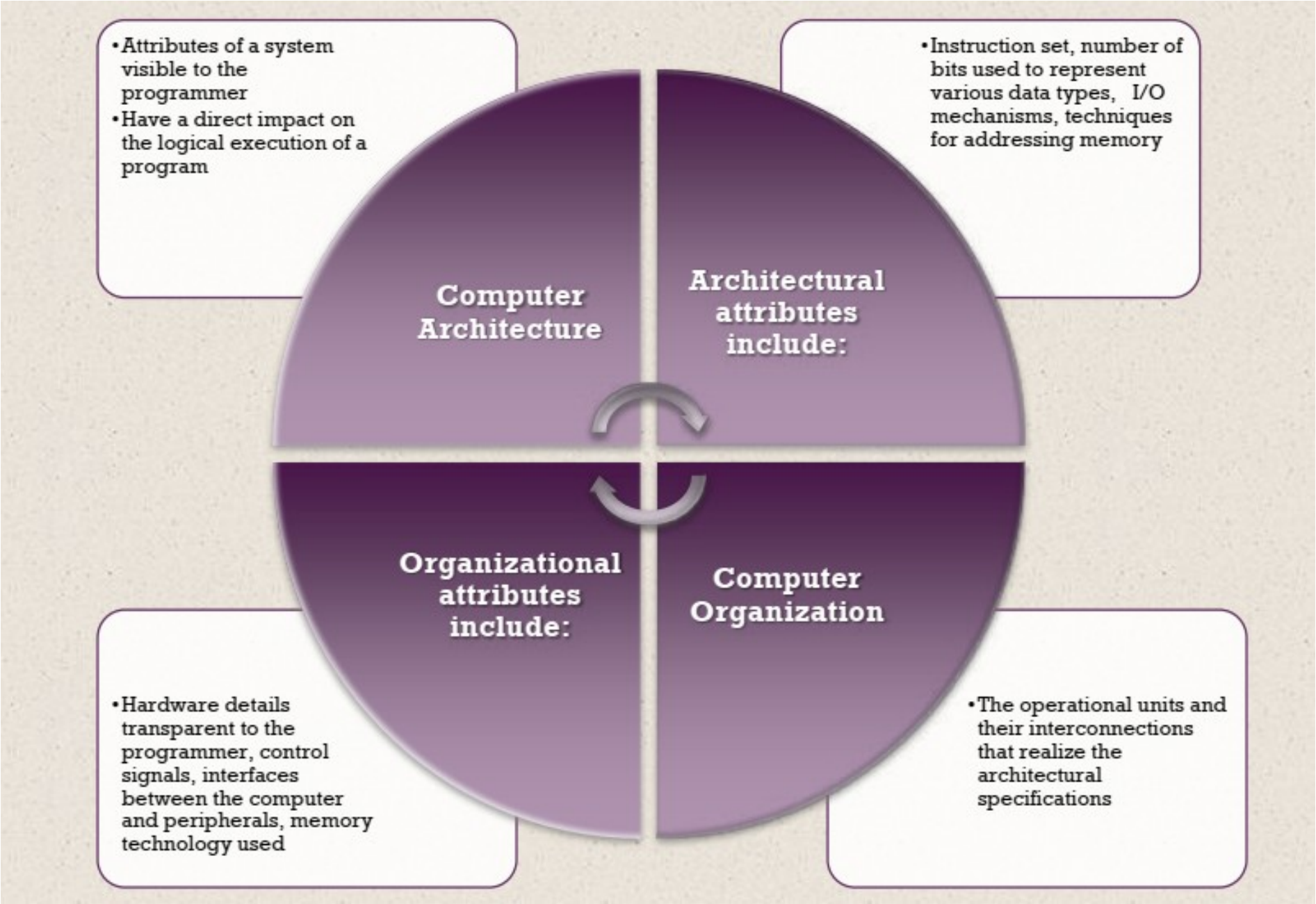
binayak.ojha@acem.edu.np

Department of Electronics and Computer Engineering

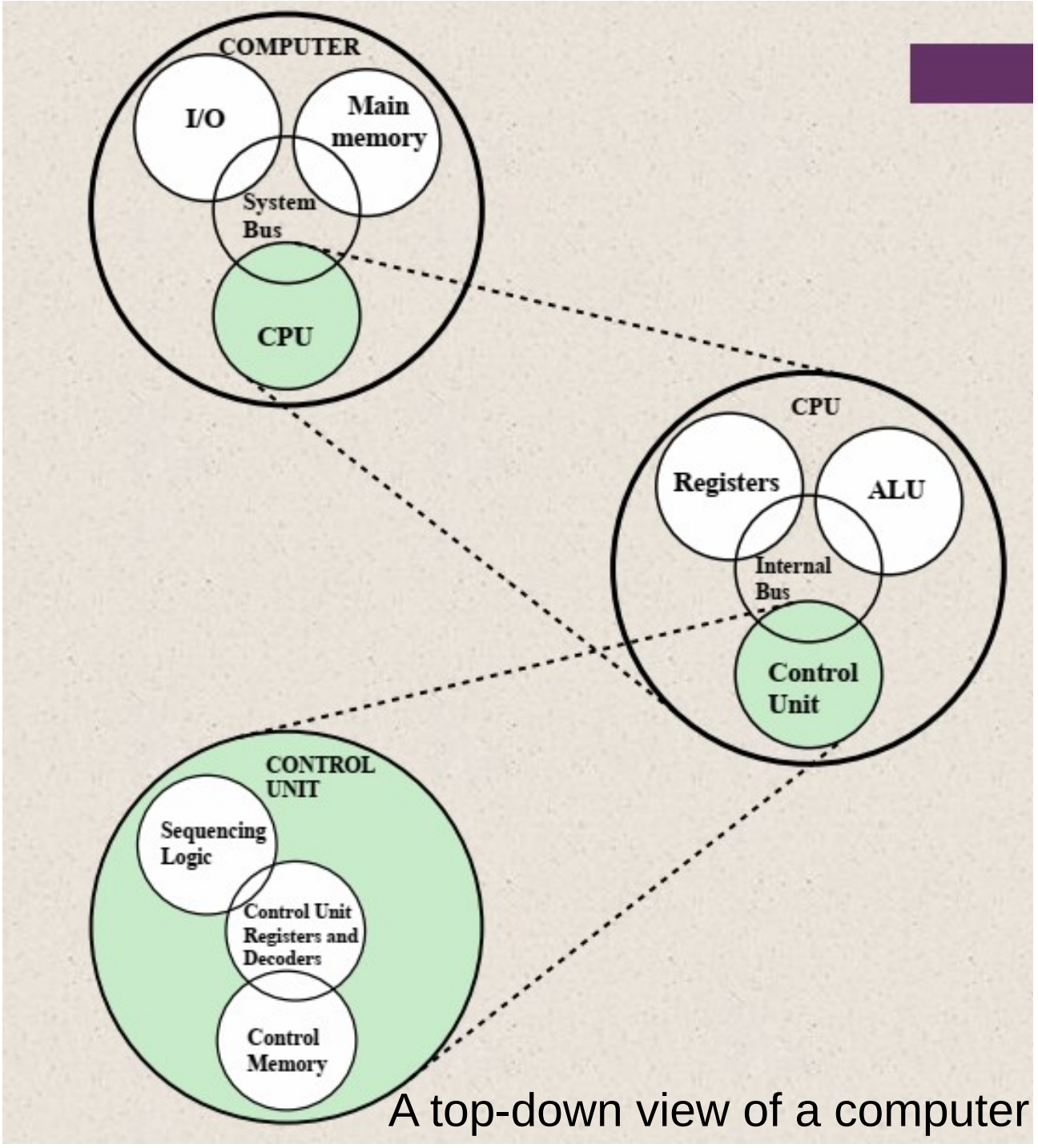
Evolution of Generation of Computers



Computer Organization and Architecture



Structure of a computer



A top-down view of a computer

Structure of a computer

There are four main structural components of the computer:

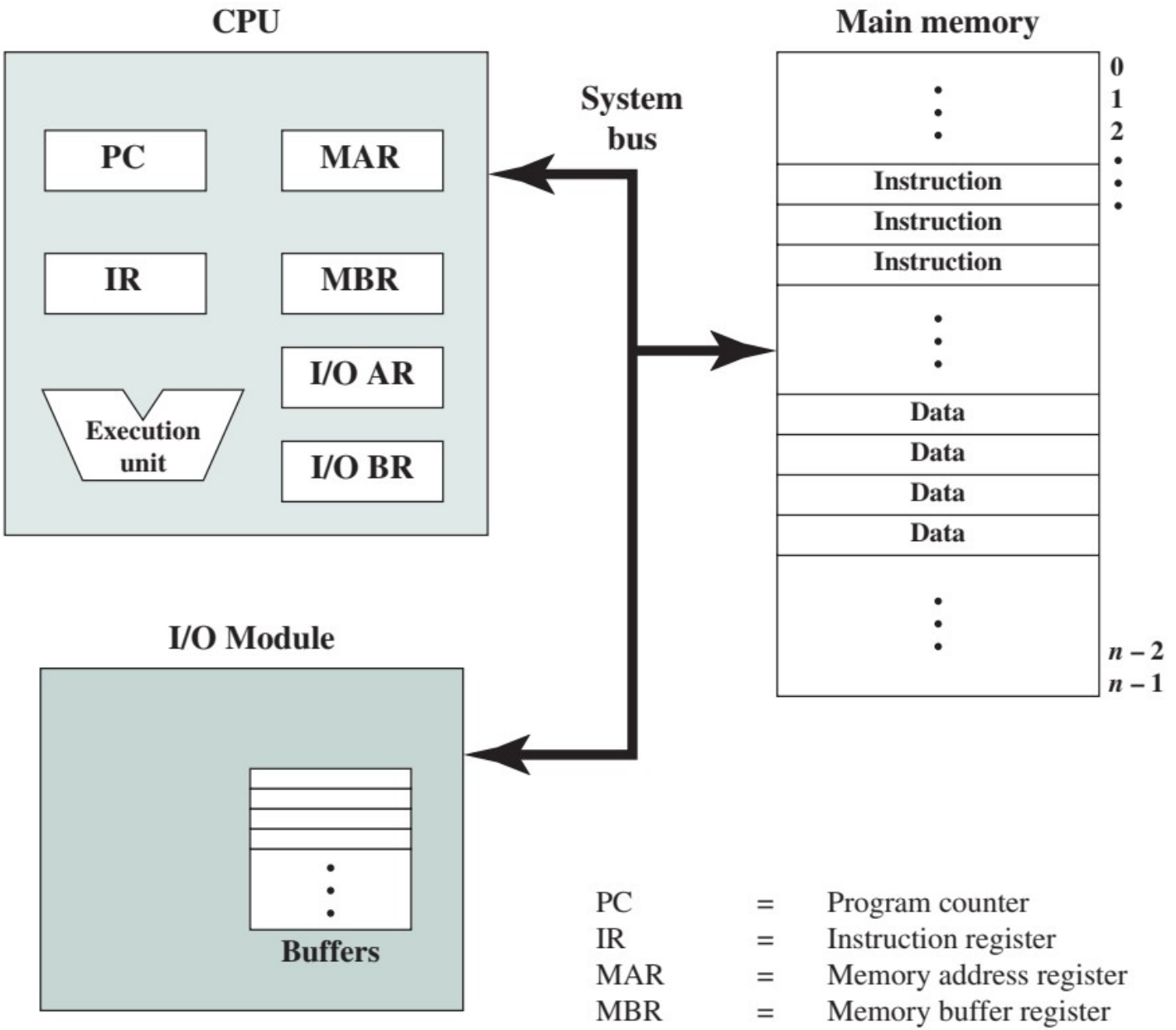
i. **CPU** → Controls the operation of the computer and performs its data processing functions

ii. **Main memory** → Stores data

iii. **I/O** → moves data between the computer and its external environment

iv. **System interconnection/System Bus** → mechanism of communication among CPU, main memory and I/O

Computer components: top-level view



PC = Program counter
IR = Instruction register
MAR = Memory address register
MBR = Memory buffer register
I/O AR = Input/output address register
I/O BR = Input/output buffer register

Major structural components of CPU

i. **Control Unit** → Controls the operation of CPU and hence the computer

ii. **Arithmetic and Logic Unit (ALU)** → Performs the computer's data processing function

iii. **Registers** → Provide storage internal to the CPU

iv. **CPU interconnection** → Some mechanism that provides for communication among the control unit, ALU and registers

Multicore computer structure

CPU

- Portion of the computer that fetches and executes instructions
- Consist of an ALU, a control unit and registers
- Referred as a proccessor in a system with a single processing unit

Core

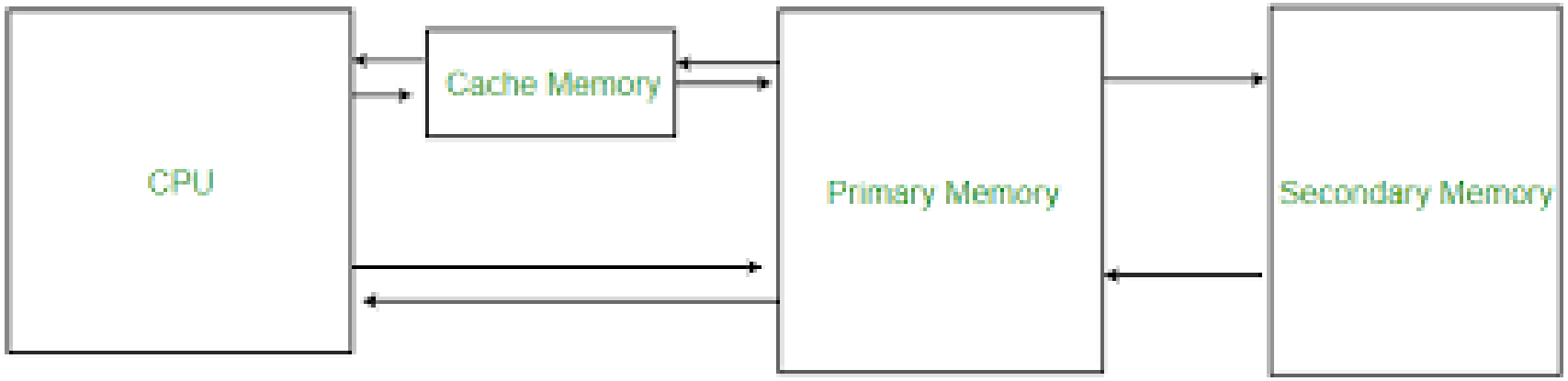
- An individual processing unit on a processor chip
- Mab be equivalent in functionality to a CPU on single CPU system
- Specialized processing units are also referred to as cores.

Processor

- A phyiscal piece of silicon containing one or more cores.
- Component of the computer that interprets and executes instructions
- Referred to as a multicore processor if it contains multiple cores.

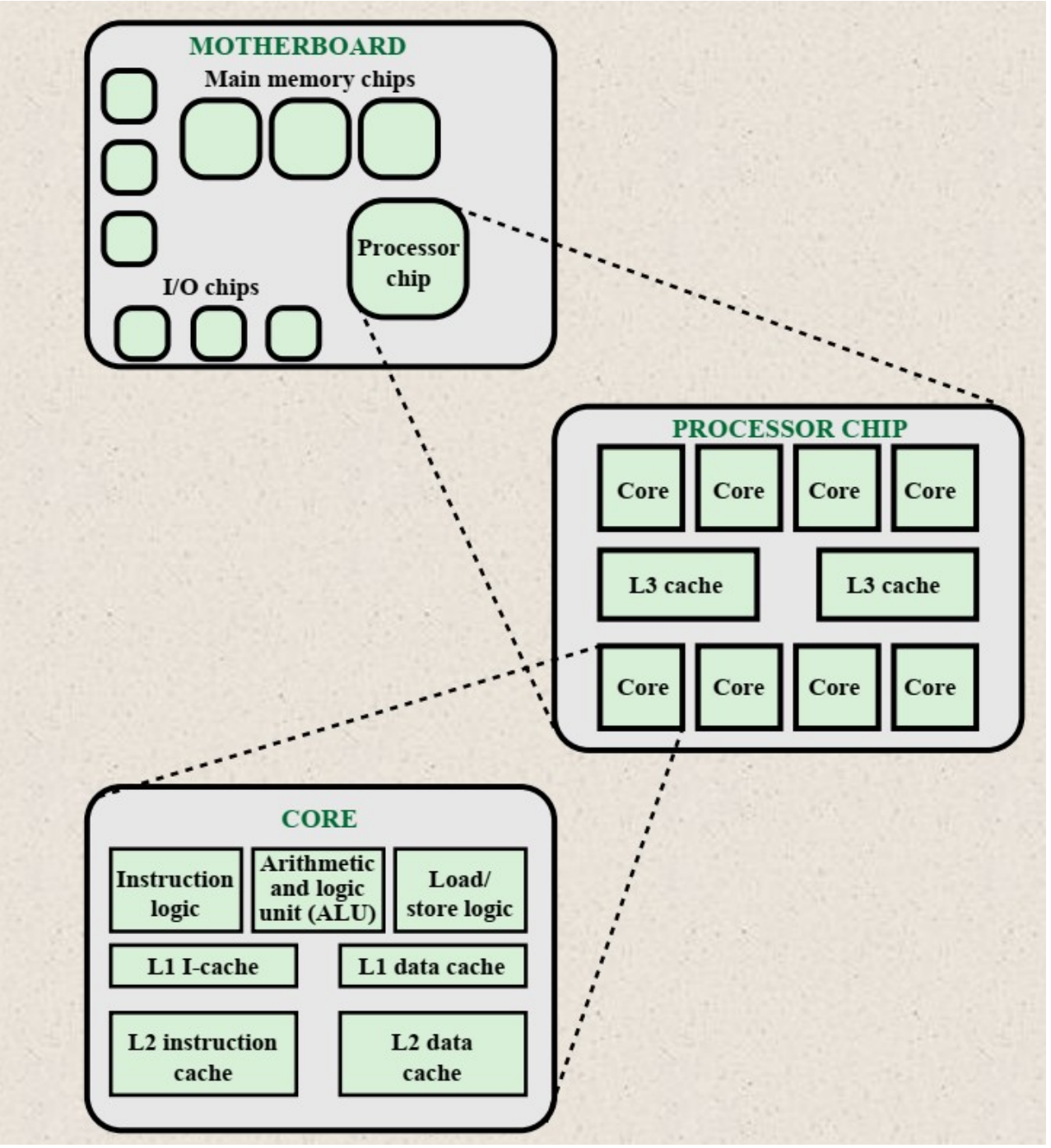
Cache Memory

- Are the multilayers of memory between the processor and main memory
- Smaller and faster than main memory.



- Used to speed up the memory access by placing in the cache data from main memory that is likely to be frequently used.
- A greater performance improvement may be obtained by using multiple levels of cache.

Major elements of multicore computer



Computer performance assessment

- Meaningful performance comparisons between processors are difficult because raw speed alone does not determine performance.
- Traditional metrics are often used to estimate processor performance, such as:
  - Clock rate
  - Instruction count
  - Cycles per instruction (CPI)
  - Execution time

Clock Speed

- Executing an instruction is broken into several discrete steps (fetch, decode, load/store, execute).
- Because of these multiple steps, most instructions require several clock cycles to complete.
- Different instructions take different numbers of cycles—some only a few, others many.
- With pipelining, multiple instructions are processed simultaneously at different stages.
- Therefore, clock speed alone is not a reliable measure of processor performance.
- Overall performance depends on factors like instruction complexity, pipeline efficiency, and architecture—not just MHz/GHz numbers.

Clock Speed

- The speed of the processor is dictated by the pulse frequency produced by the clock.

- Measured in cycles per second or Hertz (Hz)

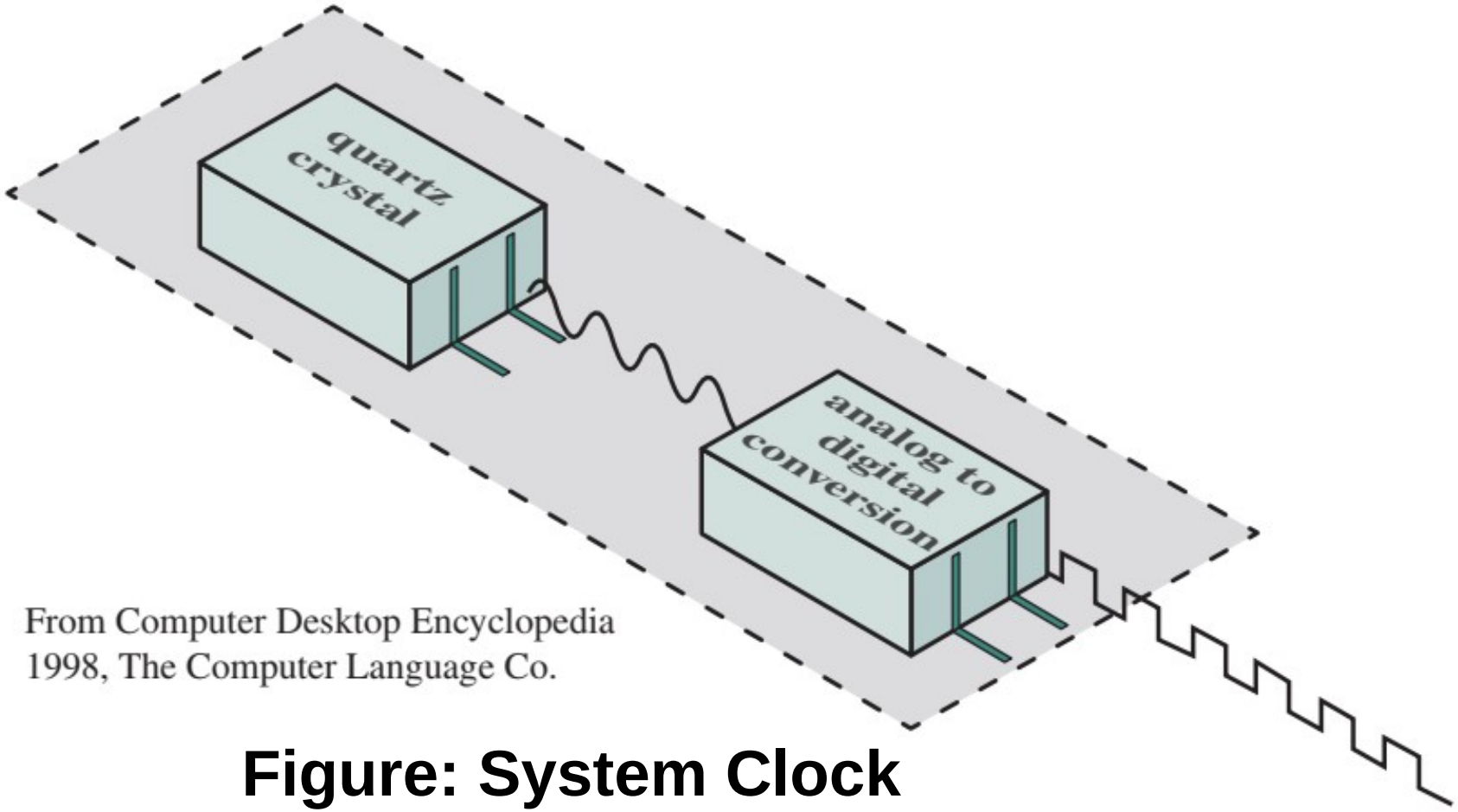


Figure: System Clock

Instruction execution rate

Average cycles per Instruction (CPI) for a program is given by

$$CPI = \frac{\sum_{i=1}^{n}(CPI_i \times I_i)}{I_c}$$

Processor time T required to execute a given program is given by

$$T = I_c \times CPI \times \tau$$

Considering the fact that during the execution of an instruction, part of the work is done by the processor, and part of the time a word is being transferred to or from memory, we can refine the preceding equation as

$$T = I_c \times [p + (m \times k)] \times \tau$$

Instruction execution rate

Where,
p is the number of processor cycles needed to decode and execute the instruction,
m is the number of memory references needed, and
k is the ratio between memory cycle time and processor cycle time.

- The five performance factors in the preceding equation (I_c, p, m, k, t) are influenced by four system attributes:

	I_c	p	m	k	τ
Instruction set architecture	X	X			
Compiler technology	X	X	X		
Processor implementation		X			X
Cache and memory hierarchy				X	X

An X in a cell indicates a system attribute that affects a performance factor.

Millions of instructions per second (MIPS)

We can express the MIPS in terms of the clock rate and CPI as follows;

$$\text{MIPS rate} = \frac{I_c}{T \times 10^6} = \frac{f}{CPI \times 10^6}$$

Millions of floating-point operations per second (MFLOPS)

$$\text{MFLOPS rate} = \frac{\text{Number of executed floating} - \text{point operations in a program}}{\text{Execution time} \times 10^6}$$

CPU execution time

CPU execution time = Instruction count x cycles per Instruction x Clock cycle time

Or,

CPU execution time = I_c x CPI/ Clock rate

Amdahl's Law

$$\text{Speedup} = \frac{\text{Time to execute program on a single processor}}{\text{Time to execute program on } N \text{ parallel processors}}$$

$$= \frac{T(1-f) + Tf}{T(1-f) + \frac{Tf}{N}} = \frac{1}{(1-f) + \frac{f}{N}}$$

Amdahl's Law

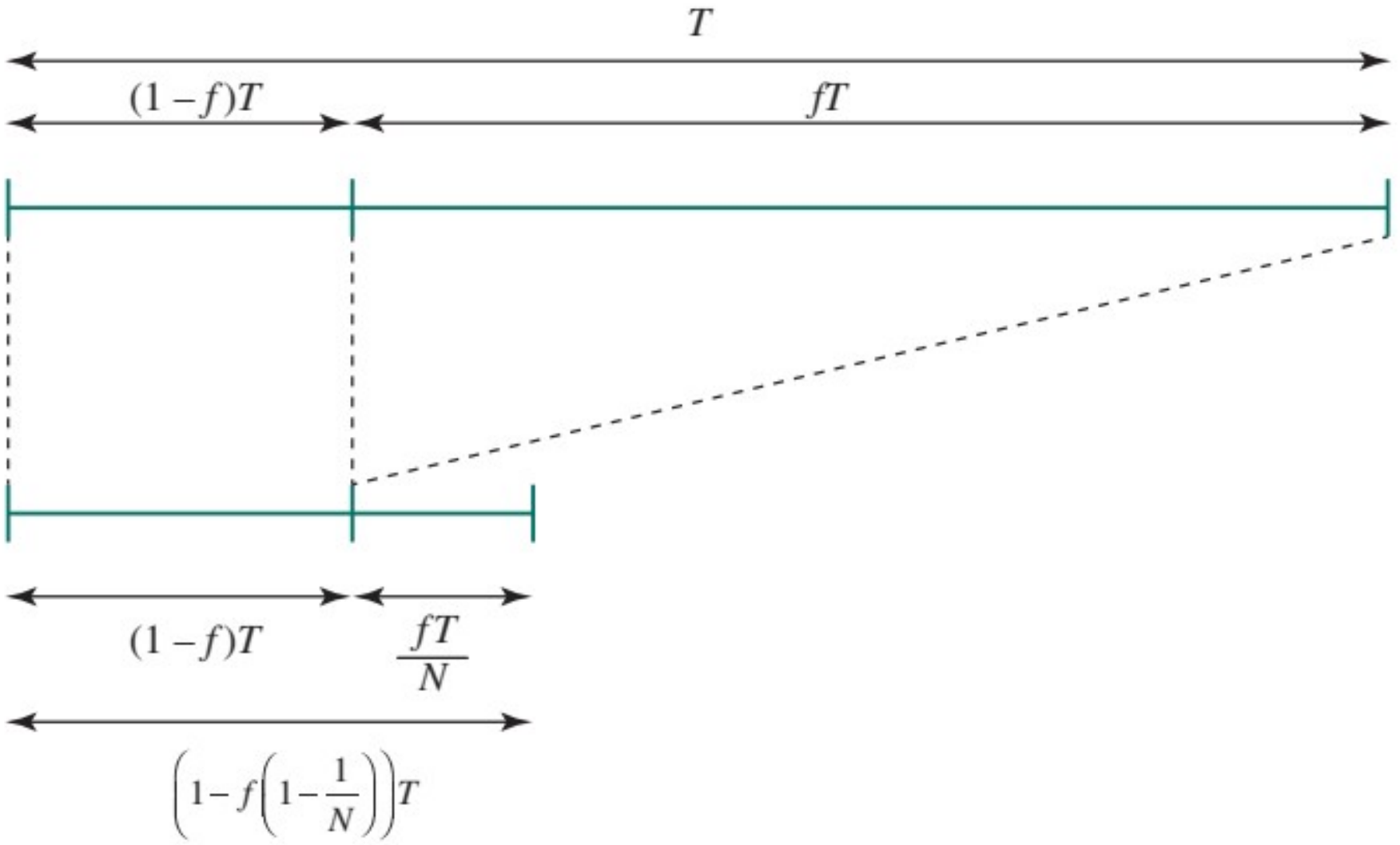
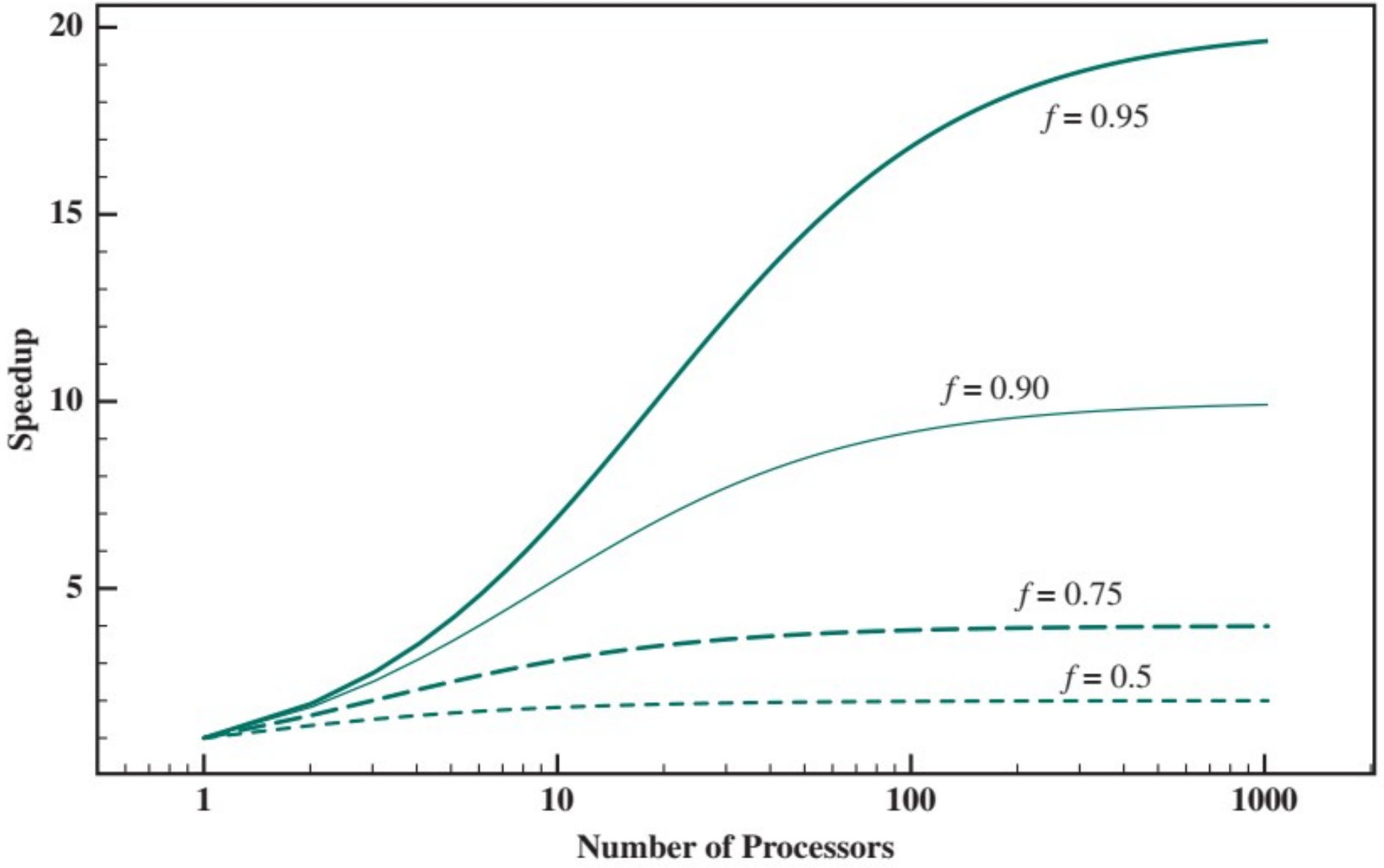


Illustration of Amdahl's Law

Amdahl’s Law



Amdahl’s Law for Multiprocessors

Amdahl’s Law

$$\text{Speedup} = \frac{1}{(1 - f) + \frac{f}{SU_f}}$$

Four basic functions of a computer

i. Data processing
ii. Data storage
    - Short-term
    - Long-term
iii. Data movement
    - I/O → when data are received or delivered to peripheral devices
    - Data communication → when data are moved over longer distances, to or from a remote device
iv. Control → Control Unit

Computer Function

Instruction fetch and execute

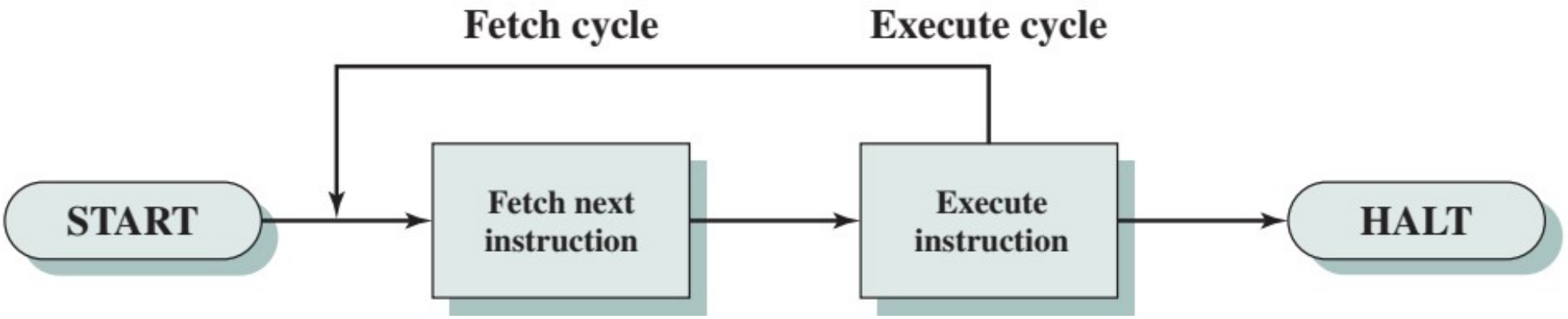
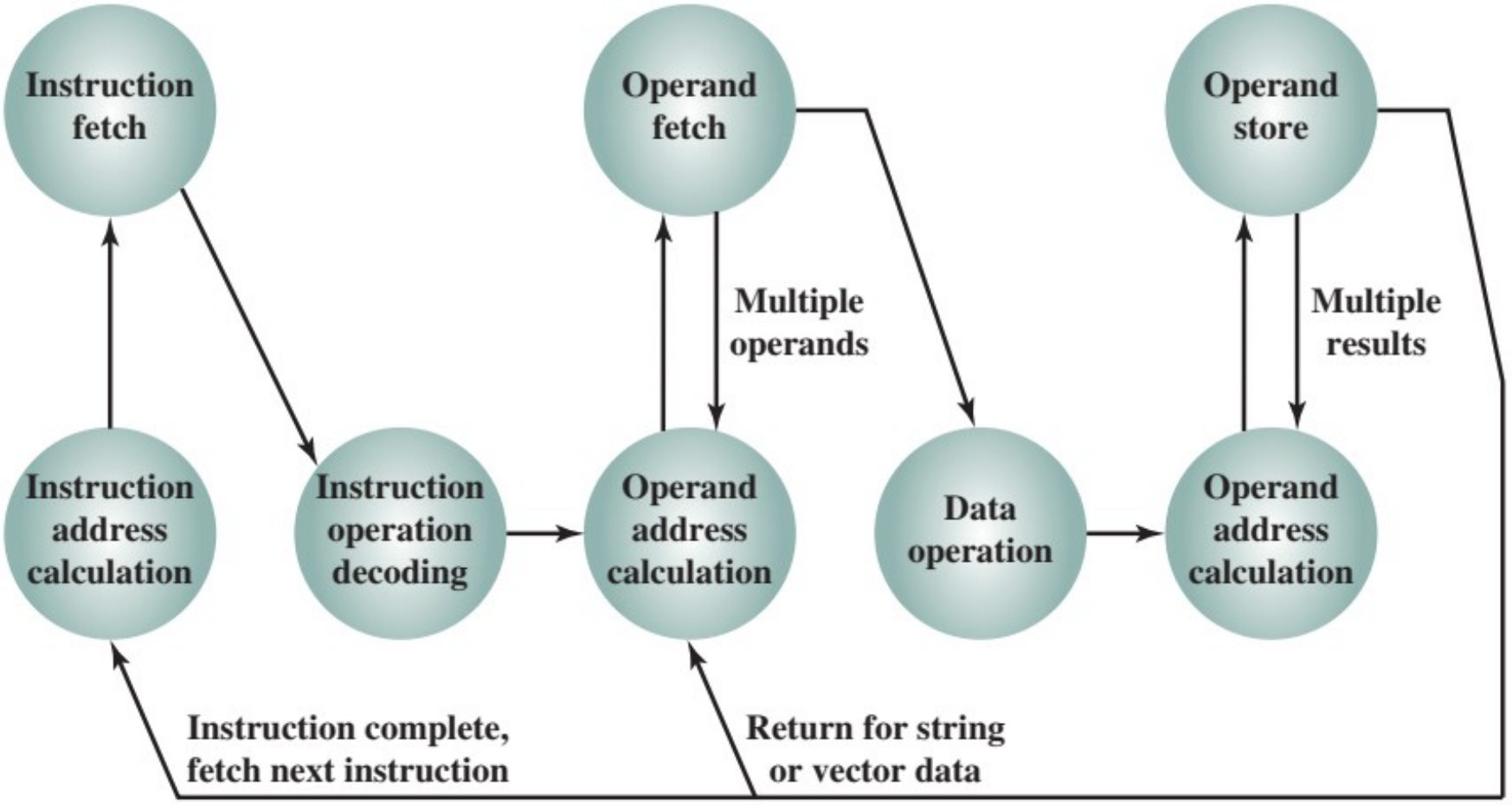


Fig: Basic Instruction Cycle

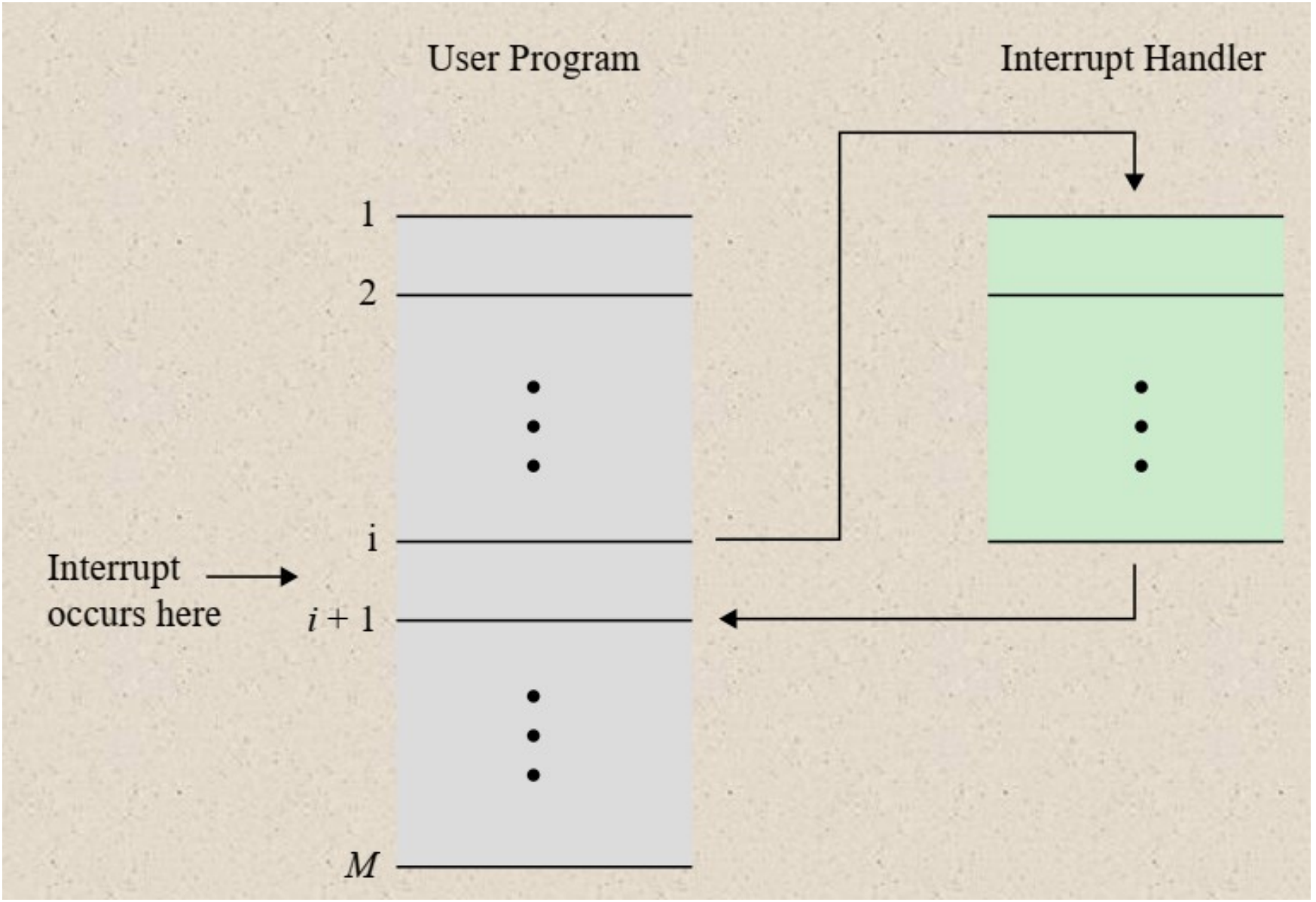
Instruction cycle state diagram



Interrupt

- Mechanism by which other modules like I/O, memory may interrupt the normal processing of the processor
- It is a signal generated by hardware or software that requires the processor’s immediate attention, temporarily suspending the currently executing program.
- Interrupts allow the CPU to respond efficiently to high-priority events, such as I/O completion, errors, or timer expirations, without constant polling.
- Interrupt Handling Process
  - When an interrupt occurs during instruction i, the processor finishes executing instruction i.
  - The processor saves the Program Counter (PC) (which points to the next instruction, i+1) and other critical context (registers, flags).
  - The PC is then loaded with the address of the ISR.
  - The ISR executes to service the interrupt.
  - After completion, the processor restores the saved context and resumes execution from instruction i+1.

Interrupt



Transfer of control via Interrupt

Interrupt

- **Types of interrupt**
  - **Software interrupts**
    - Generated by software or the system.
    - Also known as traps or exceptions.
    - Triggered by a specific "interrupt instruction" in the code.
    - The processor stops its current task and switches to an interrupt handler.
    - The interrupt handler resolves the issue or performs a task, then returns control to the application.
  - **Hardware interrupts**
    - Devices are connected to an Interrupt Request Line.
    - All devices share a single request line.
    - A device generates an interrupt by closing its associated switch.
    - The interrupt line (INTR) reflects the logical OR of all individual device requests.

Interrupt

- Classes of interrupt

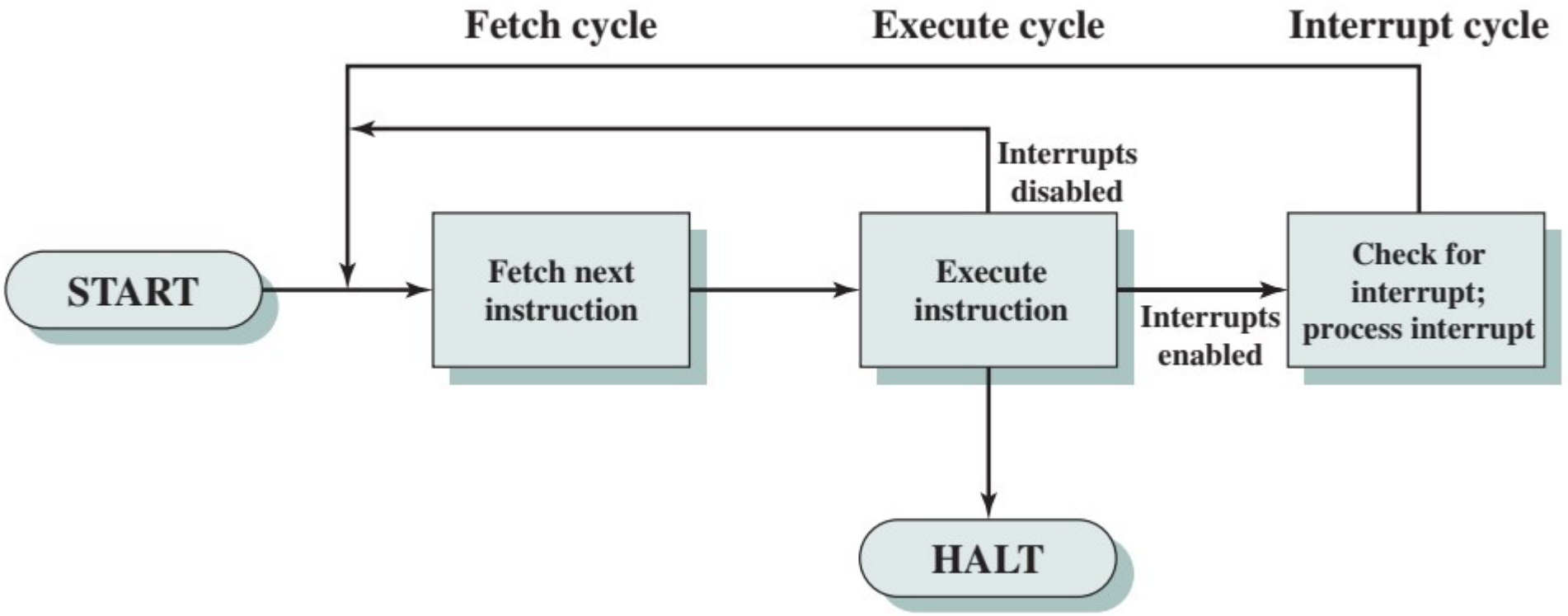
  i. Program

  ii.Timer

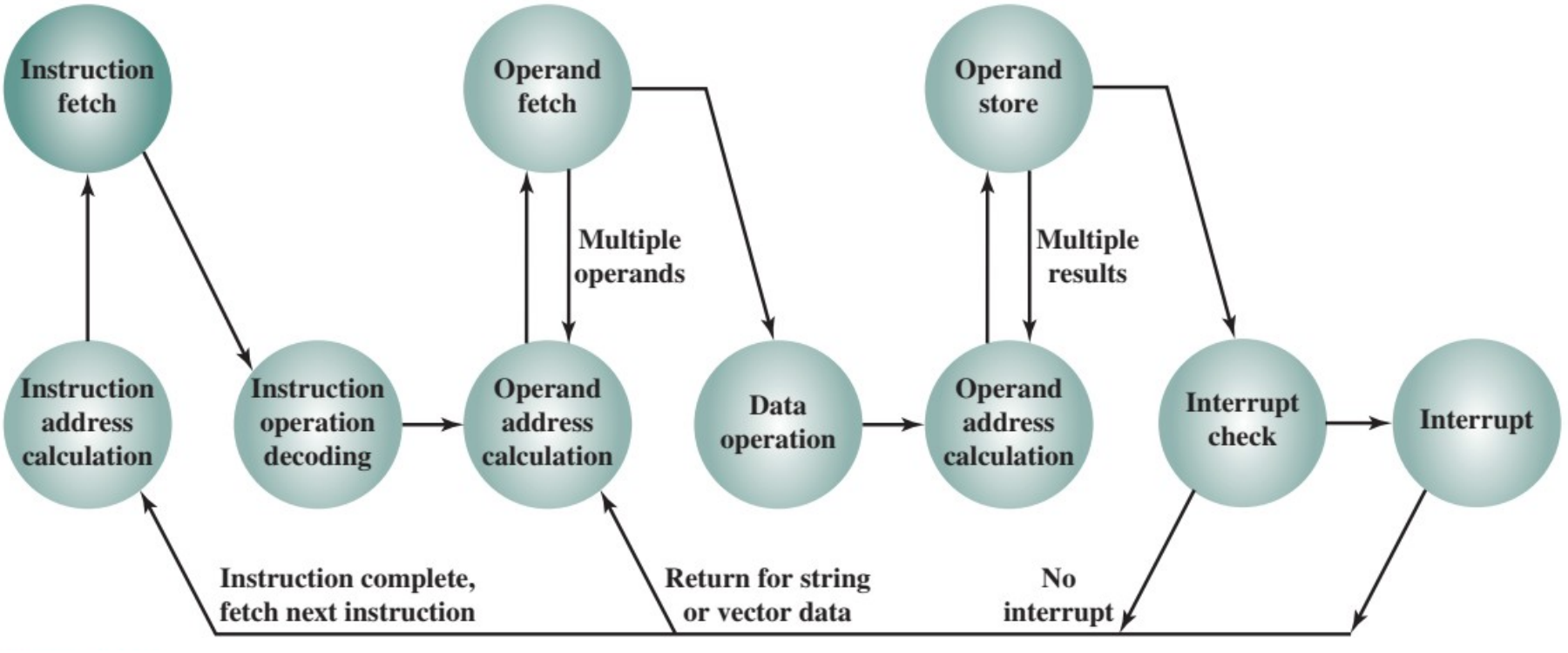
  iii.I/O

  iv.Hardware failure

Instruction cycle with interrupts

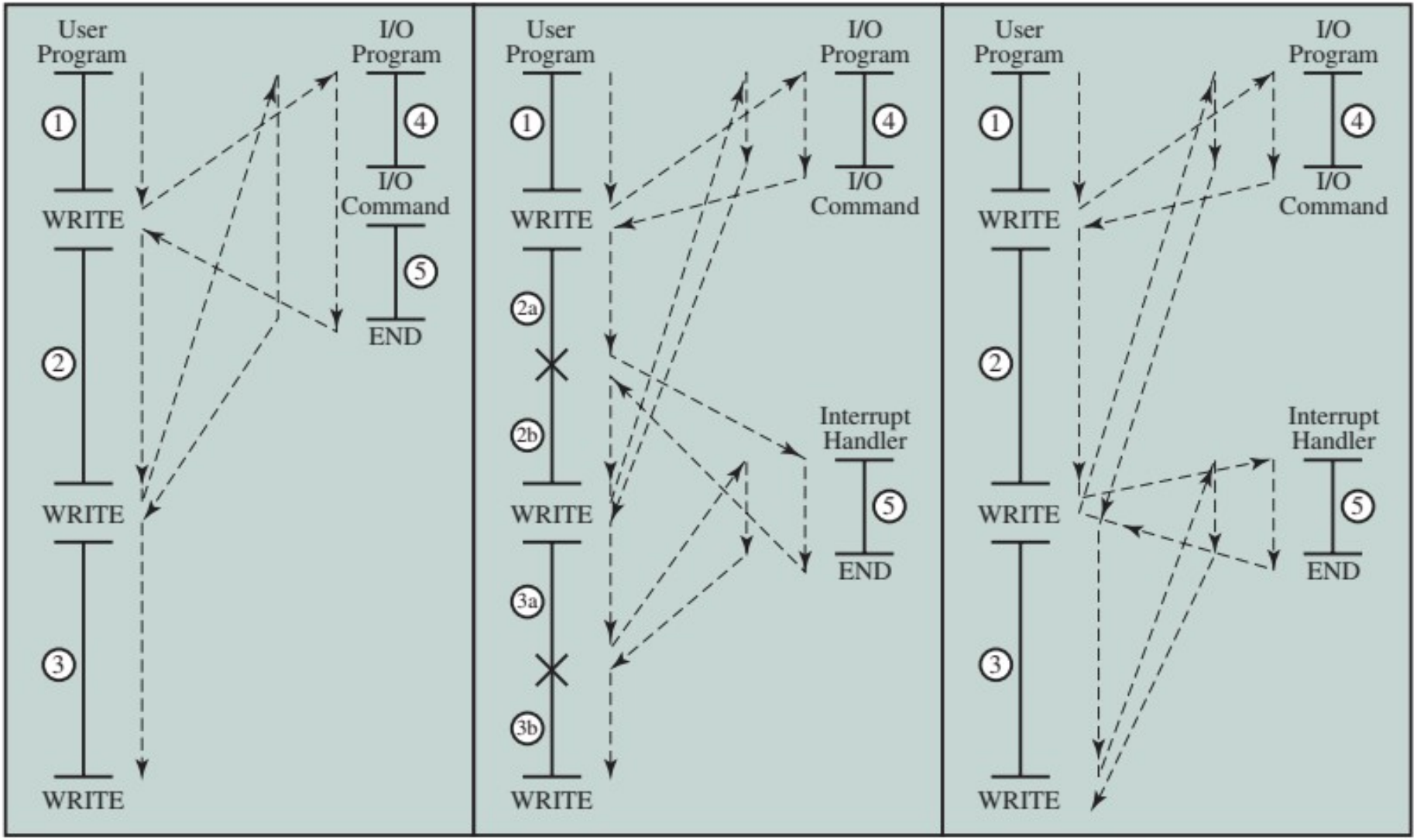


Instruction cycle state diagram with interrupts



Interrupt

- Program flow of control without and with interrupts



(a) No interrupts (b) Interrupts; short I/O wait (c) Interrupts; long I/O wait

× = interrupt occurs during course of execution of user program

Multiple Interrupt handling approaches

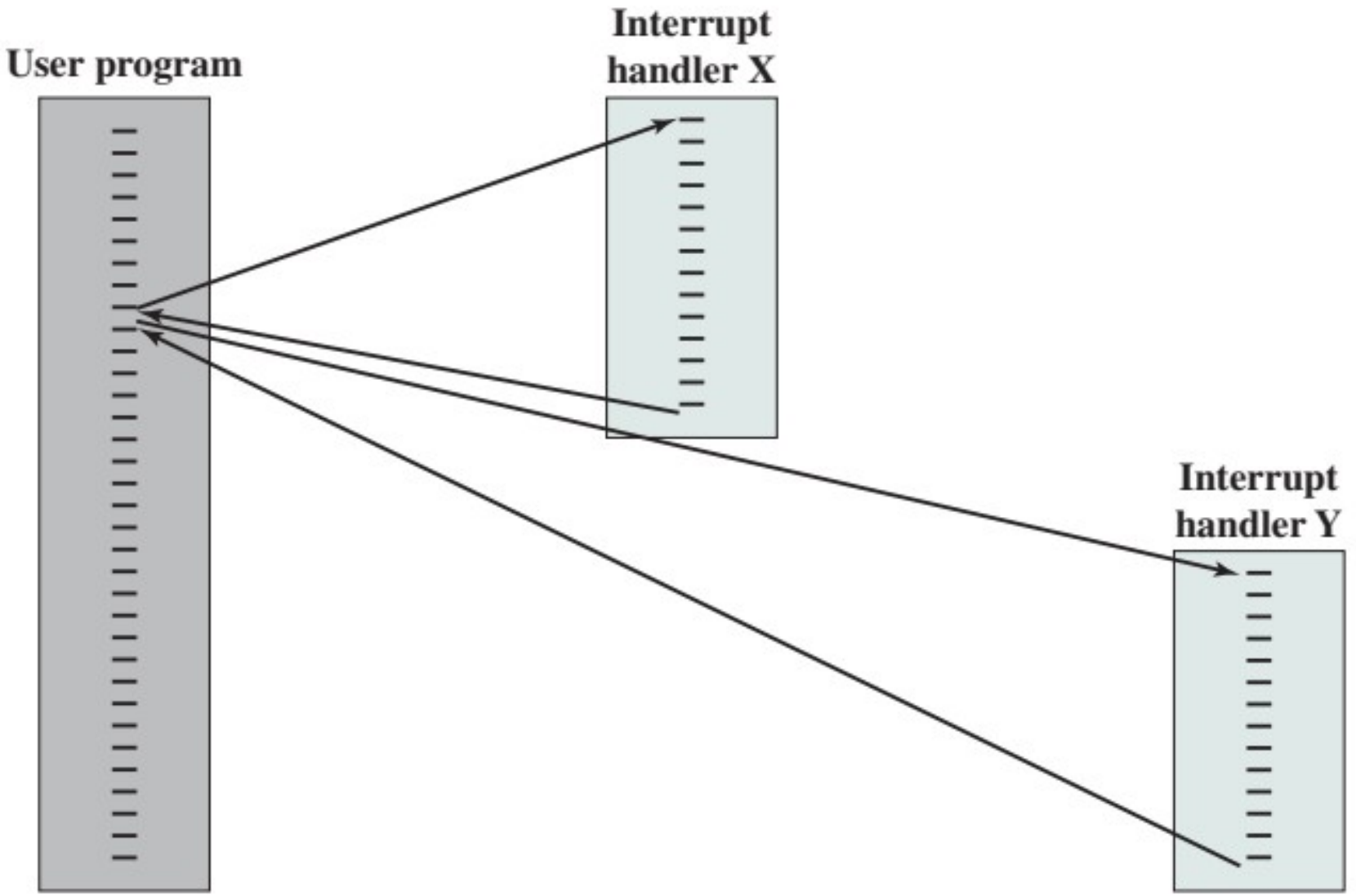
Approach 1: Disable interrupt

- processor will ignore further interrupts whilst processing one interrupt
- Interrupts remain pending and are checked after first interrupt has been processed
- Interrupts handled in sequence as they occur

Approach 2: Define priorities

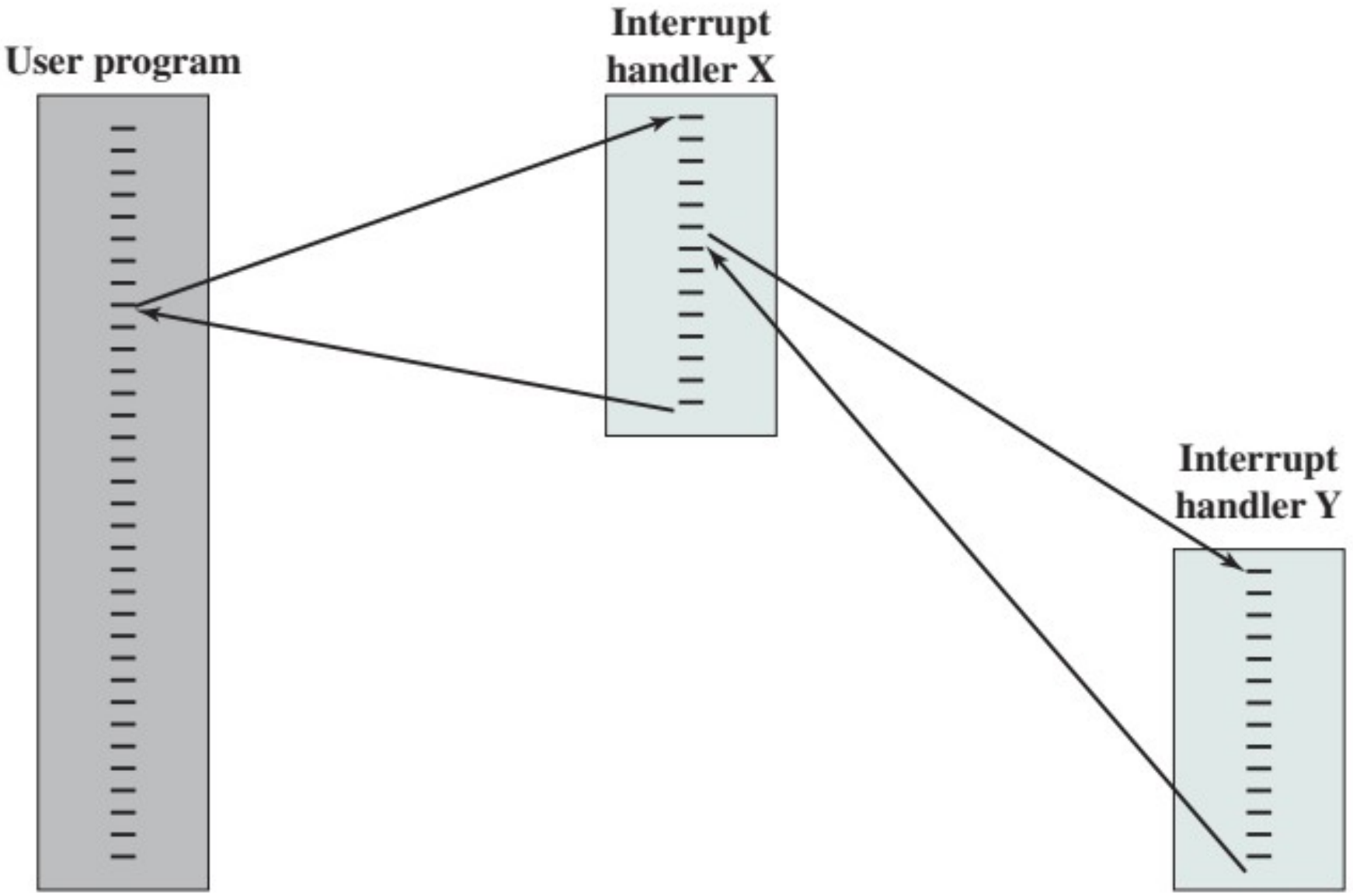
- Low priority interrupts can be interrupted by higher priority interrupts
- When higher priority interrupt has been processed processor returns to previous interrupt

Approach 1:Disable interrupt



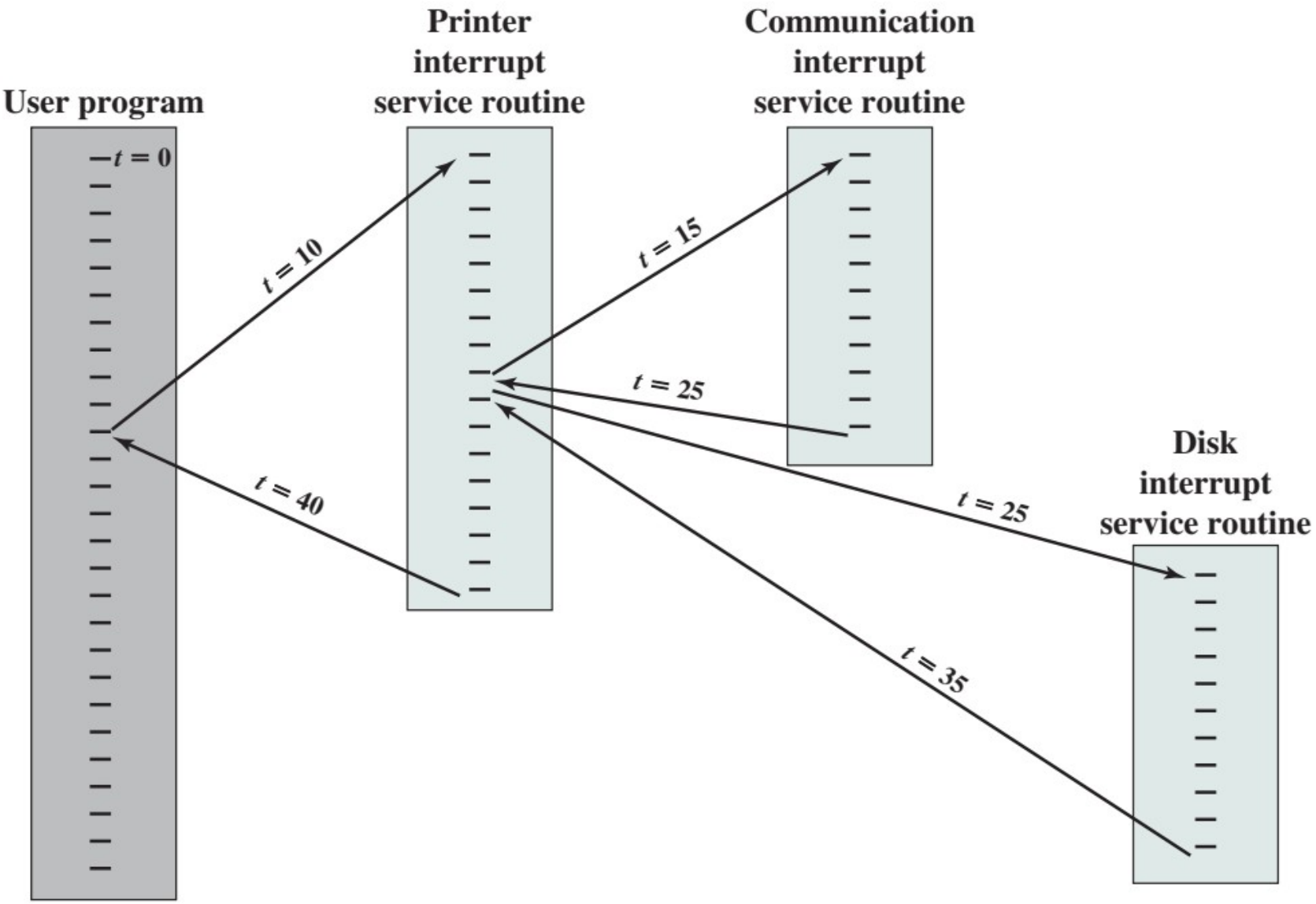
Sequential interrupt processing

Approach 2:Define priorities



Nested interrupt processing

Approach 2:Define priorities



Example time sequence of multiple interrupt

Interconnection Structures

- The collection of paths connecting the various modules is called the interconnection structure

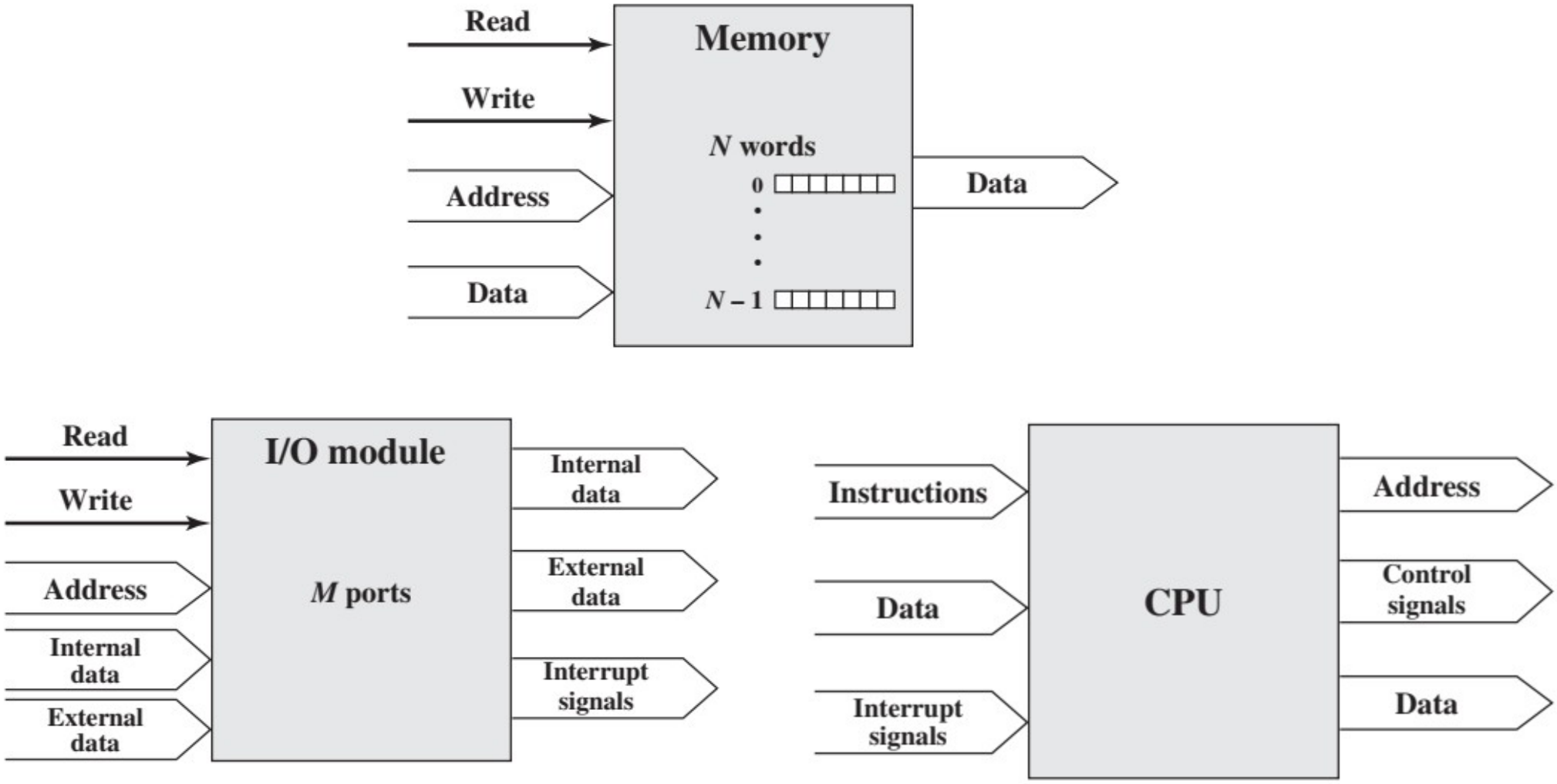


Fig: Major forms of input and output for different modules

Bus Interconnection

- A bus that connects major computer components (processor, memory, I/O) is called a system bus.
- Any bus the lines can be classified into three functional groups: data, address, and control lines.
- Data lines provide a a path for moving data among system modules. → Collectively called as data bus
- Address lines are used to designate the source or destination of the data on the data bus.→ Collectively called as address bus
- Control lines are used to control the access to and the use of the data and address lines.

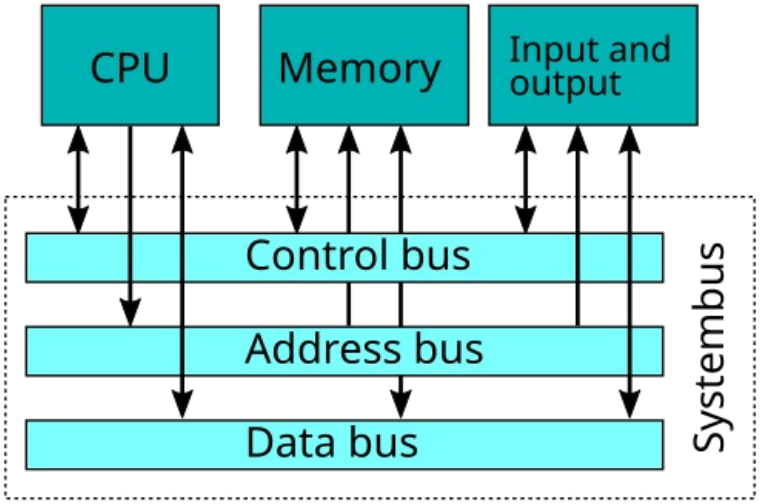


Fig: Bus Interconnection

Elements of bus design

1. Bus types

- Dedicated → separate data and address lines
- Multiplexed → Shared lines; e.g. address vaild or data valid control line

2. **Bus arbitration**

- Only one module may control bus at one time

- Types

  a) Centralized arbitration
    - Single hardware device controls the bus access

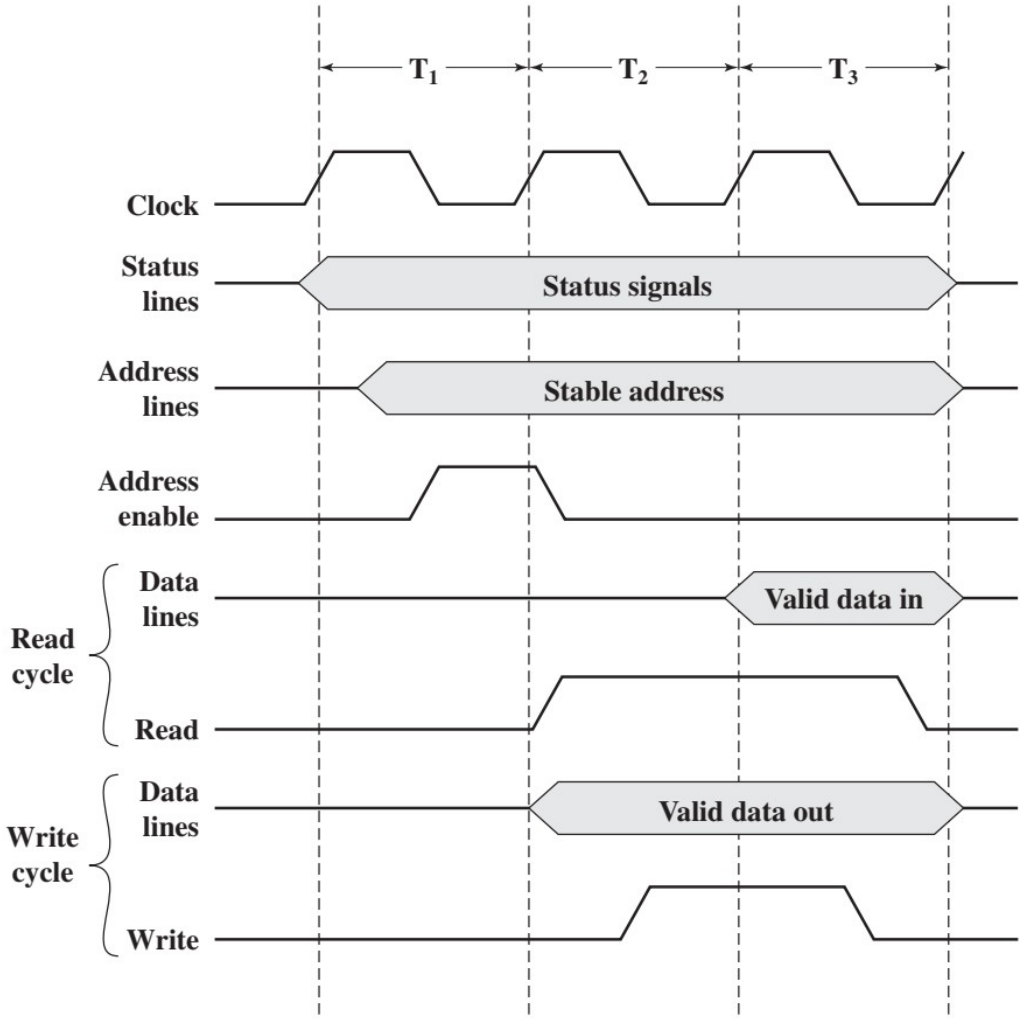
  b) Distributed arbitration
    - Each module may claim the bus based on control logic

Elements of bus design

3. Timing

- Cordination of events on bus

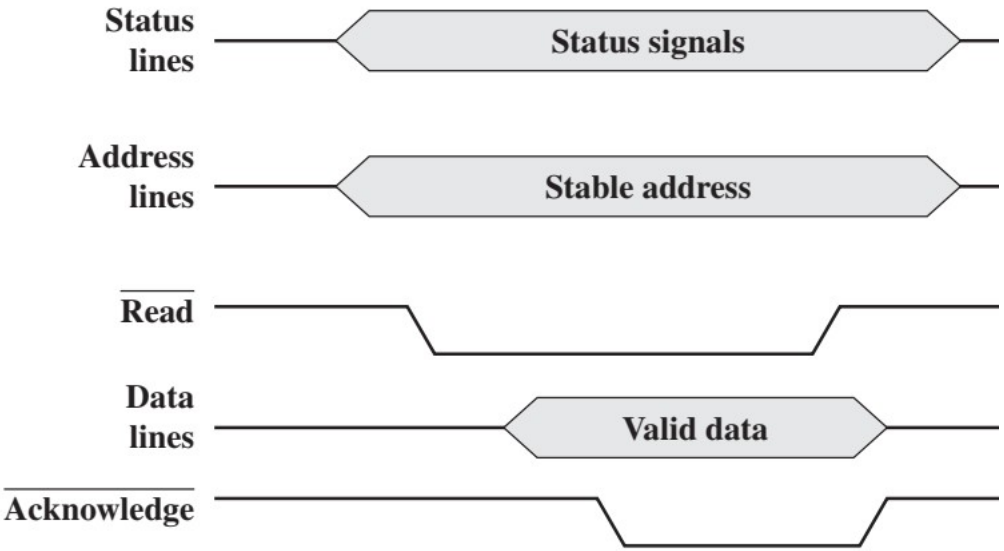
a) Synchronous timing → events determined by clock signals



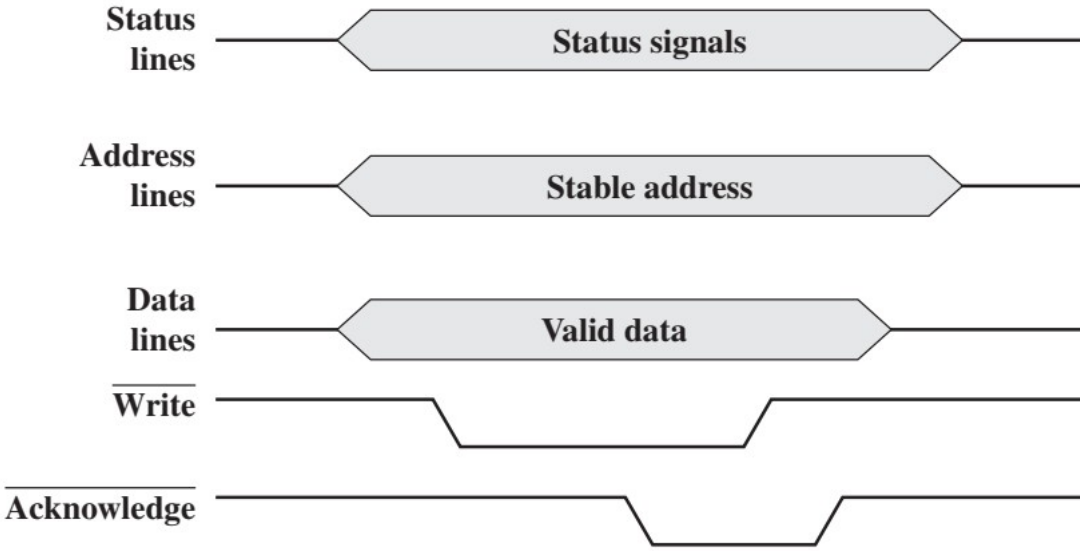
Elements of bus design

3. Timing

a) Asynchronous timing → each bus event occurs in response to previous event, not to clock.



System bus read cycle



System bus write cycle

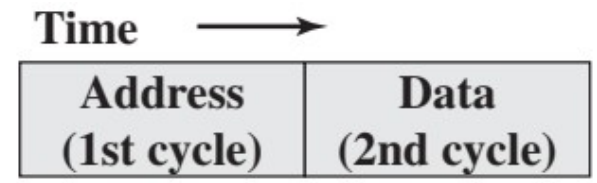
Elements of bus design

4. Bus width

- Wider the data bus, greater the number of bits transferred at one time.
- Wider the address bus, greater the range of locations that can be referenced.

Elements of bus design

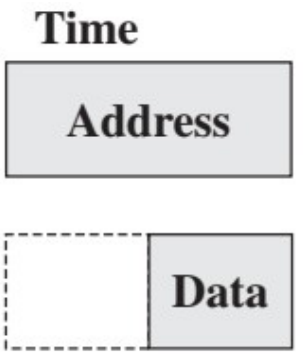
5. Data transfer types



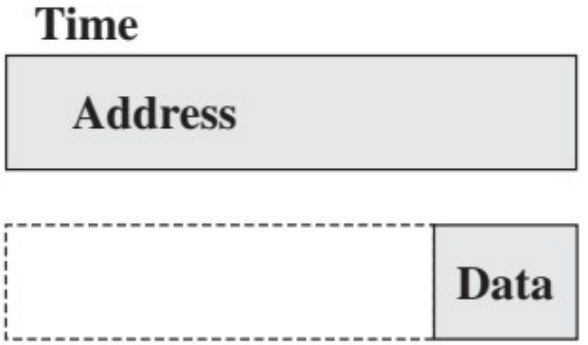
Write (multiplexed) operation

Read (multiplexed) operation

Read-modify-write operation



Write (non-multiplexed) operation

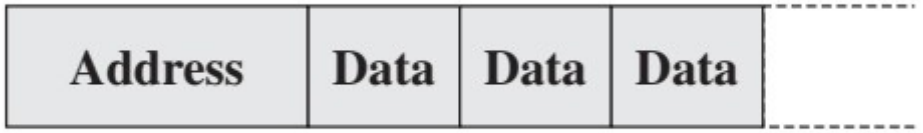


Read (non-multiplexed) operation

Elements of bus design

5. Data transfer types

Read-after-write operation



Block data transfer

Peripheral Component Interconnect

- PCI (Peripheral Component Interconnect) is a high-bandwidth, processor-independent bus.
- Can function as a mezzanine bus or a peripheral bus.
- Optimized for high-speed I/O subsystems such as graphics adapters, network cards, and disk controllers.
- **Performance**
  - Supports up to 64 data lines at 66 MHz.
  - Maximum raw data transfer rate:528 MB/s or 4.224 Gbps.
  - Usually implemented as a 32-bit bus
  - Runs at 33 MHz or 66 MHz
  - At 33 MHz and a 32-bit bus, data rate is 133 MB/s
  - Delivers better performance than many earlier bus standards.

Peripheral Component Interconnect

Design advantages

- High speed is not the only advantage:
  - Requires few chips, making it cost-effective.
  - Supports other buses attached to the PCI bus.
- Uses synchronous timing.
- Employs a centralized arbitration scheme.

Peripheral Component Interconnect

Standardization and Compatibility

- Developed by Intel in 1990 for Pentium-based systems.
- Intel released PCI patents to the public domain.
- Managed and maintained by the PCI Special Interest Group (SIG).
- Ensures vendor-independent compatibility.
- Widely adopted in PCs, workstations, and servers.

System Support

- Designed for both single-processor and multiprocessor systems.
- Provides a general-purpose set of bus functions.

Peripheral Component Interconnect

System Architecture Integration

Single-processor systems:

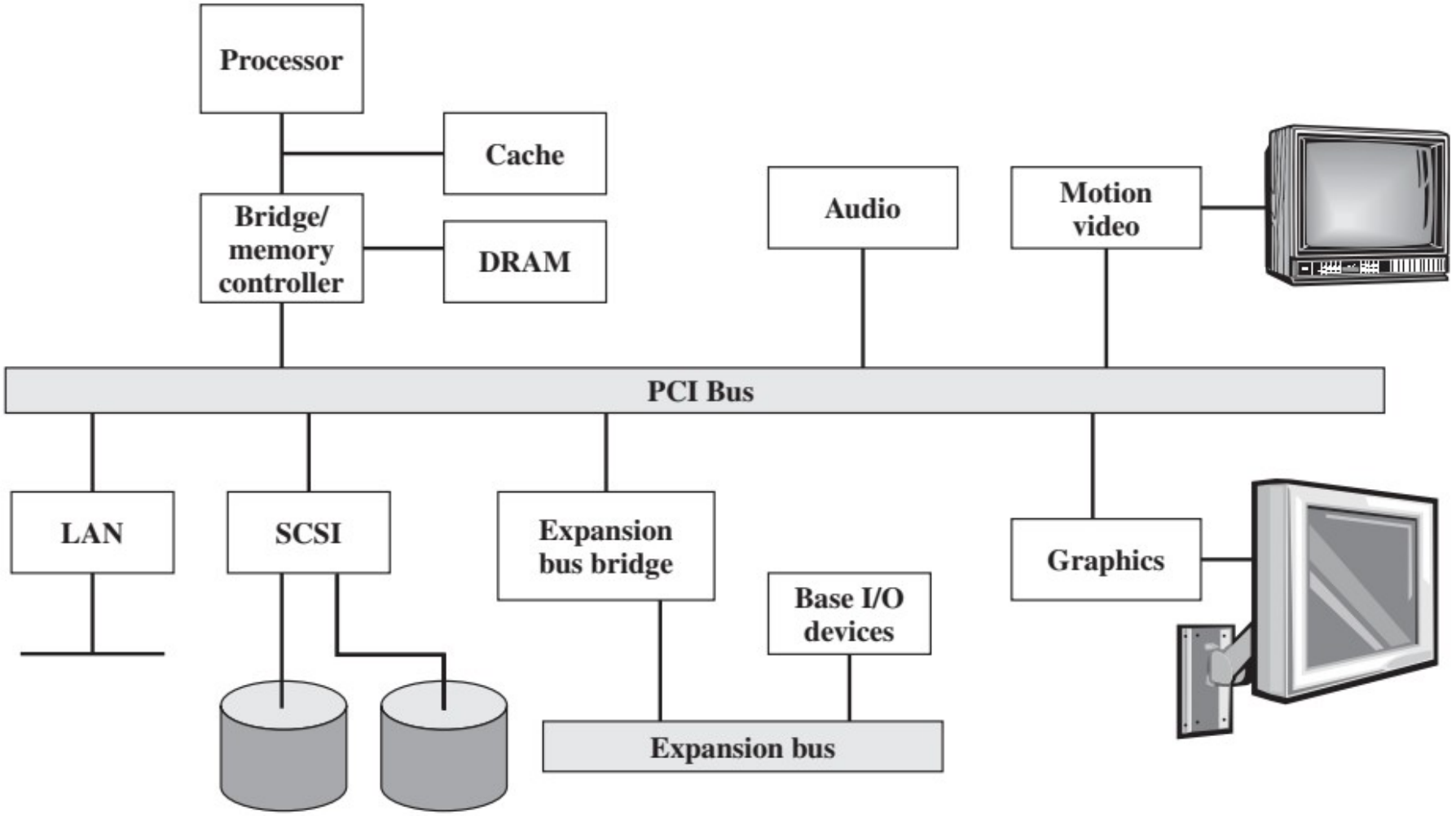
- Use a combined DRAM controller and PCI bridge.
- Bridge buffers data and allows PCI speed to differ from processor speed.

Multiprocessor systems:

- One or more PCI buses connected via bridges to the system bus.
- System bus connects processors, cache, main memory, and PCI bridges.
- Bridges maintain PCI independence from processor speed while enabling high data throughput.

Peripheral Component Interconnect

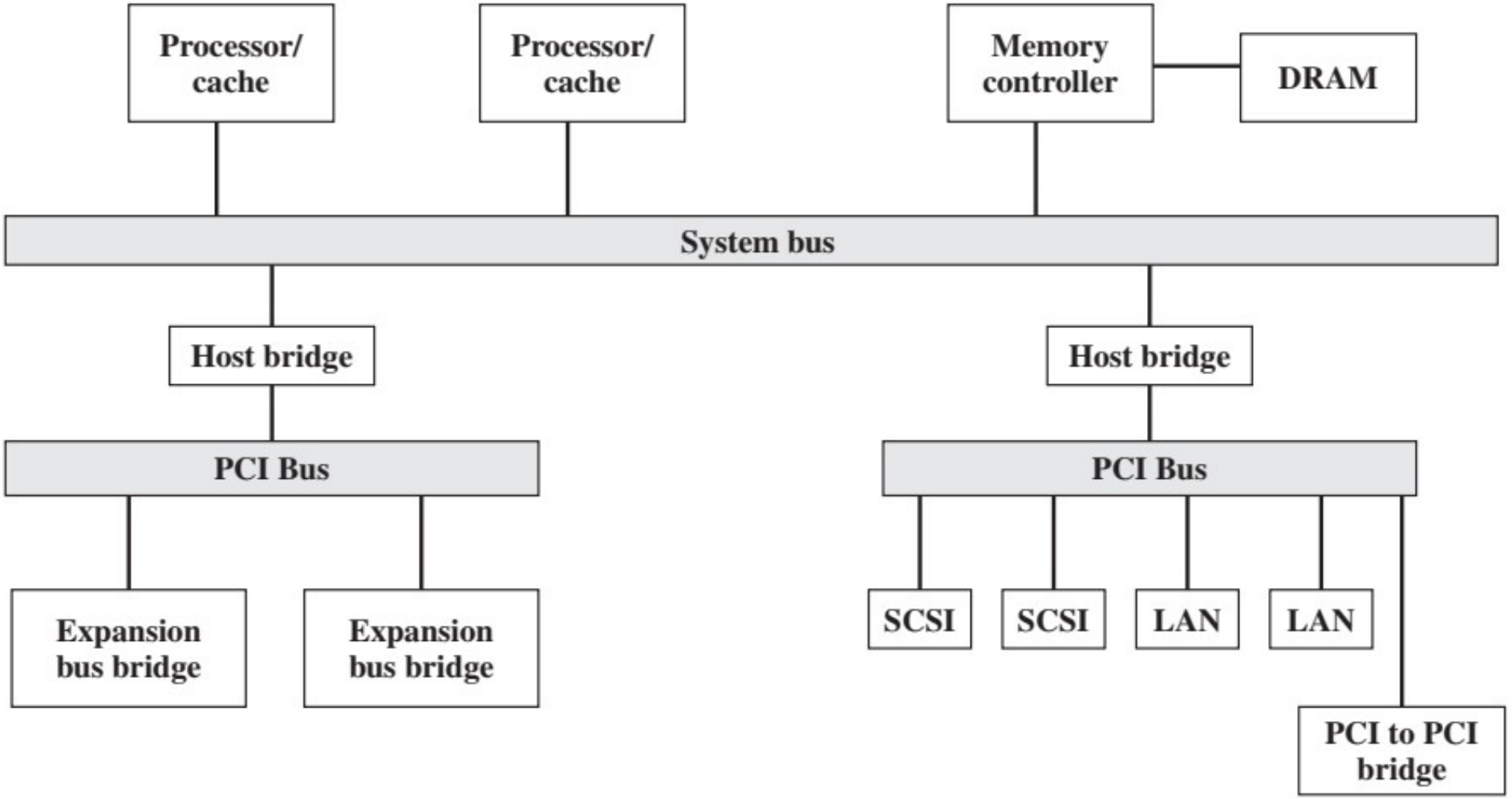
Example- PCI configuration



Typical desktop system

Peripheral Component Interconnect

Example- PCI configurations



Typical server system

Peripheral Component Interconnect

Bus structure

- PCI may be configured as a 32- or 64-bit bus.
- There are 49 mandatory signal lines for PCI, which are divided into the following functional groups:
  - System pins (including clock and reset pins)
  - Address and data pins
  - Interface control pins
  - Arbitration pins
  - Error reporting pins

Peripheral Component Interconnect

Bus structure

- PCI specification defines 51 optional signal lines , divided into the following functional groups:
  - Interrupt pins
  - Cache support pins
  - 64-bit bus extension pins
  - JTAG/boundary scan pins

Peripheral Component Interconnect

PCI commands

- Interrupt Acknowledge
- Special Cycle
- I/O Read
- I/O Write
- Memory Read
- Memory Read Line
- Memory Read Multiple
- Memory Write
- Memory Write and Invalidate
- Configuration Read
- Configuration Write
- Dual address Cycle